

Algorithms Problem Solving

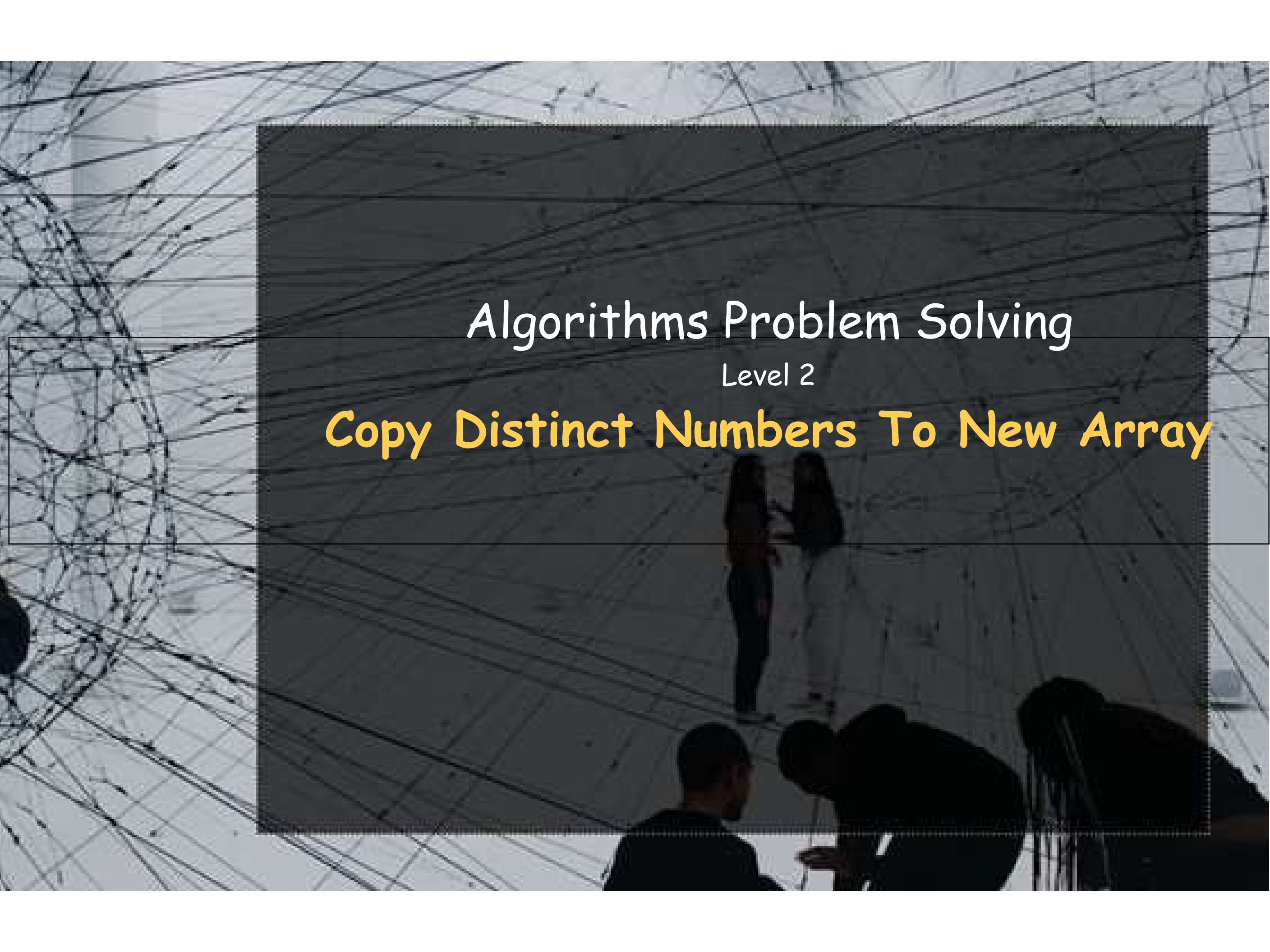
Level 2

Copy Distinct Numbers To New Array

Algorithms Problem Solving

Level 2

Copy Distinct Numbers To New Array



Definition

- ◆ This program fills an array with numbers that contain repeated values.
- ◆ It checks each number to see whether it already exists in the second array.
- ◆ Only distinct numbers are supposed to be copied to the second array.
- ◆ ``IsNumberInArray()`` is used to check whether a number already exists.
- ◆ ``AddElementToArray()`` adds a new number to the second array.

Explanation

- ◆ `FillArray()` creates the following array:
10 10 10 50 50 70 70 70 70 90
- ◆ `CopyDistinctNumbers()` loops through every element in `arr1`.
- ◆ For each number, it checks whether that number already exists in `arr2`:
if (`!IsNumberInArray(arr2, arr1Length, arr1[i])`)
- ◆ The `!` means "not".
- ◆ Therefore, if the number is not already in `arr2`, it is added:
`AddElementToArray(arr1[i], arr2, arr2Length);`
- ◆ For example, the first `10` is not in `arr2`, so it is added.
- ◆ The next `10` should already be found in `arr2`, so it should not be added again.
- ◆ The same process is repeated for `50`, `70`, and `90`.

Steps in main()

1. Fill `arr1` with the predefined numbers.
2. Set `arr2Length` to `0`.
3. Loop through every element of `arr1`.
4. Check whether the current number already exists in `arr2`.
5. If it does not exist, add it using `AddElementToArray()`.
6. Increase `arr2Length`.
7. Continue until all elements of `arr1` have been checked.
8. Print the distinct numbers stored in `arr2`.

💡 Example

◆ `Array 1` :

10 10 10 50 50 70 70 70 70 90

◆ The process should be:

10 → not found → add

10 → already exists → skip

10 → already exists → skip

50 → not found → add

50 → already exists → skip

70 → not found → add

70 → already exists → skip

70 → already exists → skip

70 → already exists → skip

90 → not found → add

◆ Therefore, the distinct values should be:

10 50 70 90

➡ Result

The intended result is:

Array 1 Elements: 10 10 10 50 50 70 70 70 70 90

Array 2 Distinct numbers: 10 50 70 90

📌 Note

- ◆ `FindNumberPositionInArray()` searches for a number and returns its index.
- ◆ `IsNumberInArray()` converts that result into `true` or `false`:
 `return FindNumberPositionInArray(arr, arrLength, number) != -1;`
- ◆ `CopyDistinctNumbers()` uses `!IsNumberInArray()` because we only want numbers that are not already present.
- ◆ `AddElementToArray()` stores the new distinct number and updates `arr2Length`.
- ◆ The important concept here is filtering duplicate values while building a new array.

⊘ Warning

- ◆ There are two important bugs in the current code.
- ◆ First, this line:
 if (!IsNumberInArray(arr2, arr1Length, arr1[i]))
 should use `arr2Length`, not `arr1Length`:
 if (!IsNumberInArray(arr2, arr2Length, arr1[i]))
- ◆ `arr2` does not contain `arr1Length` elements. It only contains the elements that have already been copied.
- ◆ Second, `main()` prints `arr1` instead of `arr2`:
 PrintArray(arr1, arr2Length);
- ◆ It should be:
 PrintArray(arr2, arr2Length);
- ◆ Also, the comment says "distinct numbers" but the example output `10 10 10 50` is incorrect. True distinct values are `10 50 70 90`.
- ◆ The array sizes are `10`, so this particular program is designed around 10 elements.



Comparison

`arr1`

- ✓ Contains the original numbers.
- ✓ Can contain duplicate values.
- ✓ Example: `10 10 10 50 50 70 70 70 70 90`.

`arr2`

- ✓ Stores only distinct values.
- ✓ Should not contain duplicates.
- ✓ Example: `10 50 70 90`.
- ✓ Its length is stored in `arr2Length`.

✦ Summary

- ✓ Start with an array containing duplicate numbers.
- ✓ Check each number against the elements already stored in `arr2`.
- ✓ Use `IsNumberInArray()` to detect duplicates.
- ✓ Add the number only when it does not already exist.
- ✓ Use `arr2Length` when searching inside `arr2`.
- ✓ Print `arr2`, not `arr1`.
- ✓ The correct distinct result is `10 50 70 90`.

Thank you 😊